DATA ENGINEERING REPORT · PROJECT 1

Global health: SDG 3 in Africa

A reliable PostgreSQL warehouse of WHO indicators: ingestion, quality, upsert and 2030 target analysis

LUMU BAMANA MANASSE

Data engineer

Date	October 2026
Version	1.0.0 (delivered and accepted)
Client	Initiative Baobab Santé (fictional NGO) · Engagement IBS-2026-001
GitHub repository	https://github.com/MediJr/who-sdg3-health-etl
Language	English · *version française disponible dans le dépôt*

Fictional scenario, real data: the client NGO and its staff are invented to simulate a complete professional engagement. The data comes from the WHO Global Health Observatory, extracted on 22 October 2026. This project is not commissioned, endorsed by or affiliated with the World Health Organization.

Document control

Project	Portfolio project 1: PostgreSQL warehouse of WHO health indicators
Engagement	IBS-2026-001: "Monitor SDG 3 in Africa with WHO data"
Client	Initiative Baobab Santé, maternal and child health NGO (fictional), attn. Dr Mariam Koné
Author	Lumu Bamana Manasse, data engineer
Delivery period	October 2026
Data	WHO GHO API, batch of 22 October 2026
Status	DELIVERED ACCEPTED
Source code	https://github.com/MediJr/who-sdg3-health-etl

Version history

Version	Date	Description
0.1	2026-10-22	Scoping: brief, meeting minutes, statement of work
0.9	2026-10-22	Pipeline, tests, analyses, fixes (quality, performance)
1.0.0	2026-10-22	Fresh-environment acceptance, final report (French and English)

Reading guide

Board of directors	1. Executive summary and 11. Results and recommendations
Monitoring & evaluation lead	2. Scoping, 6. Data model, 7. Data quality, 12. Limitations
Engineer taking over	4. Architecture, 5. Pipeline, 9. Operations, 10. Testing, appendices

Contents

Document control 2

1 Executive summary 4

2 Context and scoping 5
2.1 The client's real problems . . . 5
2.2 Business questions . . . 5
2.3 Technical requirements . . . 6
2.4 Scope . . . 6

3 Data sources **6**

4 Solution architecture **7**
4.1 Technology choices 8
4.2 What changed since project 0 8

5 Pipeline design **8**
5.1 Extraction 8
5.2 Transformation and quality checks 9
5.3 Loading: staging and upsert 9

6 Data model **10**

7 Data quality **10**
7.1 Coverage 10
7.2 Threshold calibration: a lesson from real-world data 10

8 Security and governance **11**

9 Operations **11**
9.1 Commands 11
9.2 Production diagnoses 12

10 Testing and acceptance **12**
10.1 Automated tests 12
10.2 Fresh-environment acceptance 12

11 Results and recommendations **13**
11.1 Q1 · Where do countries stand against the 2030 targets? 13
11.2 Q2 · Which countries will meet the targets by 2030? 15
11.3 Q3 · How large is the gap between the poorest and richest households? 17
11.4 Q4 · Are inputs associated with outcomes? 18
11.5 Recommendations to the board 19

12 Limitations and technical debt **19**

13 Lessons learned **19**

14 Roadmap **20**

A SQL queries delivered **20**

B Architecture decision records (ADR) **20**

C Glossary **21**

1 Executive summary

The need. The NGO Initiative Baobab Santé tracks African countries' progress towards the SDG 3 targets every year. Its process was manual (downloads, copy-pasting into Excel), not reproducible, and had already led to an error presented to the board: a table mixed several data categories without anyone noticing.

The solution. A Python pipeline automatically feeds a containerised **PostgreSQL** warehouse with 9 WHO indicators. The grain is explicit, quality checks quarantine invalid rows, and an **upsert** load applies WHO revisions without ever creating duplicates. The client's team queries the database in SQL.

129,368 WHO observations	**9** SDG 3 indicators	**386** locations (countries, regions, world)	**22 / 22** automated tests
0 / 0 inserts / updates on re-run	**10 s** full reload	**×15** speed-up after diagnosis	**0** personal data

Key findings for the WHO African Region (47 countries)

- **Q1 · 2030 targets:** only **5 of 47 countries** have met the under-five mortality target (25 per 1,000), and 4 the maternal mortality target (70 per 100,000). The region remains far behind the global average.
- **Q2 · Pace: no** off-target country is on track to meet the under-five mortality goal by 2030. The median country reduces it by 3.0% a year, but would need 12.4%: **four times faster**.
- **Q3 · Inequality:** a child from the poorest 20% of households is **1.7 times more likely** to die before age 5 (up to 2.5 times in Nigeria). The absolute gap is shrinking, but the relative gap has widened since 2000.
- **Q4 · Inputs: skilled birth attendance** is the input most strongly associated with low mortality (correlation of -0.75).

What this project demonstrates on the engineering side: a containerised PostgreSQL database, secrets kept out of the code, a layered model (staging, core, ops), a complete business key with `NULLS NOT DISTINCT`, bulk loading with `COPY` followed by an idempotent and measurable upsert, quality checks with quarantine, a run log, unit and integration tests, and two production diagnoses: a Docker outage and a 130-second connection brought down to zero.

2 Context and scoping

2.1 The client's real problems

Analysis of the brief showed that the client's problem was first a **data engineering** problem, and only then an analytics one:

Symptom described by the client	Technical cause
"We download the tables by hand"	No reproducible or traceable process
"The numbers never match from one year to the next"	WHO revises its estimates, and nobody keeps track of revisions
"A table mixed several categories"	**Grain error**: the same country-year value exists by sex, age and wealth quintile

Table 1: From symptom to cause

2.2 Business questions

#	Question	Decision supported
Q1	Where do African Region countries stand against the 2030 SDG 3 targets?	Diagnosis presented to the board
Q2	At their current pace, which countries will meet the targets by 2030?	Prioritising countries to fund
Q3	How large is the child mortality gap between the poorest and richest households?	Programme targeting
Q4	Are health system inputs associated with better outcomes?	Choice of levers

Table 2: Questions agreed during scoping

2.3 Technical requirements

Requirement	Technical answer
SQL access for several users	PostgreSQL 17 in Docker
Re-runnable at any time, no duplicates	Upsert on a complete business key
Latest published value, and revision date	`last_changed_at`, `last_run_id`
Explicit grain	`UNIQUE NULLS NOT DISTINCT` key on 7 columns
Checks before loading	9 rules and a quarantine table
Passwords out of the code	`.env` file excluded from Git
Automated tests	`pytest`: 17 unit tests and 5 integration tests

Table 3: Statement of work requirements and how they were met

2.4 Scope

The database holds the 9 indicators for **all countries and regions** published by WHO, so it is reusable beyond this engagement. The analyses focus on the **WHO African Region** (47 countries with data), with the global value as a benchmark.

3 Data sources

GHO resource	Content	Volume (22 Oct batch)
`/api/{indicator}`	Observations: value, uncertainty bounds, update date	129,368 rows, 9 indicators
`/api/Indicator`	Official indicator names	9 rows
`/api/DIMENSION/{type}/DimensionValues`	Location and breakdown labels	386 locations, 174 breakdowns

Table 4: WHO API resources used (OData protocol, no API key)

Code	Indicator	Rows	2030 target
MDG_0000000007	Under-five mortality (per 1,000)	64,510	≤ 25
WHOSIS_000001	Life expectancy at birth	14,112	–
WHOSIS_000002	Healthy life expectancy	14,112	–
WHOSIS_000003	Neonatal mortality (per 1,000)	12,128	≤ 12
MDG_0000000026	Maternal mortality (per 100,000)	7,878	< 70
WHS4_100	DTP3 immunisation coverage (%)	5,337	90%
GHED_CHE_pc_US_SHA2011	Health expenditure per capita (USD)	4,849	–
HWF_0001	Doctors per 10,000	3,693	–
MDG_0000000025	Skilled birth attendance (%)	2,749	–

Table 5: Indicators collected

Key exploration finding: the hidden dimension

For Senegal in 2020, under-five mortality appears **8 times with 8 different values**, from 27.9 to 61.2 per 1,000. These are not duplicates: the `Dim3` column carries the **wealth quintile** (61 per 1,000 for the poorest 20% of households, 28 for the richest 20%). This is exactly the mistake the client made the year before. The finding set the **grain** of the fact table and gave rise to question Q3.

4 Solution architecture

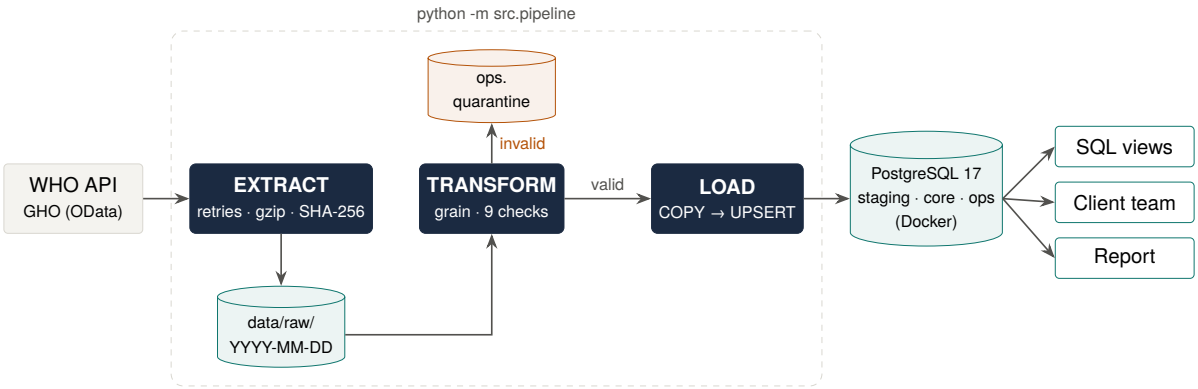


Figure 1: Architecture: one command collects, checks and loads the data into PostgreSQL

4.1 Technology choices

Need	Choice	Rationale
Database	PostgreSQL 17	Multi-user, strong constraints, native upsert, `NULLS NOT DISTINCT` (ADR 001)
Running the database	Docker Compose	Same database on every machine, one command, no manual install
Driver	psycopg 3	Modern driver, `COPY` for bulk loading
Secrets	python-dotenv + `.env`	No password in the code or in Git
HTTP retries	`urllib3.Retry`	Exponential back-off, honours the `Retry-After` header
Tests	pytest	Unit tests without a database, integration tests on a dedicated database

Table 6: Technology stack (on top of Python, pandas and matplotlib, already used in project 0)

4.2 What changed since project 0

	Project 0	Project 1
Database	DuckDB (file)	PostgreSQL (server, container)
Loading	Full refresh	Staging + upsert, data kept across runs
Idempotency	Guaranteed by replacement	**Measured**: inserted / updated / unchanged counters
Quality	Checks after the fact	Checks **before** loading, plus quarantine
Secrets	None	`.env` file
Tests	Manual acceptance	22 automated tests
Traceability	Manifest	Manifest with SHA-256, and a run log in the database

Table 7: Portfolio progression

5 Pipeline design

5.1 Extraction

- **Retries delegated to** `urllib3.Retry`: up to 4 attempts, waiting 2, 4, 8 and 16 s, on transient errors only (429, 500, 502, 503, 504) and never on a 404.
- **OData pagination followed** (`@odata.nextLink`), and **explicit failure** if the response lacks the `value` key: better to crash than to load anything.
- **Compressed raw layer** (gzip, 6.4 MB for 129,368 rows) with **atomic writes**.

- **Manifest** with the SHA-256 fingerprint of each file, proving the batch's integrity.

5.2 Transformation and quality checks

Breakdowns are mapped by their **declared type** (`Dim1Type`, `Dim2Type`...), never by position: WHO may put sex in the first or second dimension depending on the indicator. Nine rules are then applied:

#	Rule (quarantine reason)
1	Indicator not in the reference list
2	Missing or non-numeric value
3	Value outside plausibility bounds (logical limits, see section 7)
4	Inconsistent uncertainty interval (lower bound > value, or value > upper bound)
5	Invalid year
6	Unknown location
7	Unknown breakdown value
8	Unknown breakdown type
9	Duplicate business key within the batch

Table 8: Quality rules applied before loading

5.3 Loading: staging and upsert

The batch is bulk-copied into `staging.observation` (`COPY`, 0.9 s), then merged into `core.fact_observation` by a single query:

```
INSERT INTO core.fact_observation AS f (...)
SELECT ... FROM staging.observation
ON CONFLICT ON CONSTRAINT uq_observation_business_key DO UPDATE SET
    value = EXCLUDED.value, value_low = EXCLUDED.value_low, value_high = EXCLUDED.value_high,
    last_changed_at = now(), last_run_id = EXCLUDED.last_run_id
WHERE (f.value, f.value_low, f.value_high)
      IS DISTINCT FROM (EXCLUDED.value, EXCLUDED.value_low, EXCLUDED.value_high)
RETURNING (xmax = 0) AS inserted      -- true: inserted, false: updated
```

- `ON CONFLICT`: if the business key already exists, update instead of insert.
- `WHERE … IS DISTINCT FROM`: only values actually revised by WHO are touched.
- `IS DISTINCT FROM` rather than <>: it compares `NULL`s correctly, which <> does not.
- Merge, quarantine and run log are committed in **a single transaction**. The run log entry is opened separately, so that a failure remains on record.

6 Data model

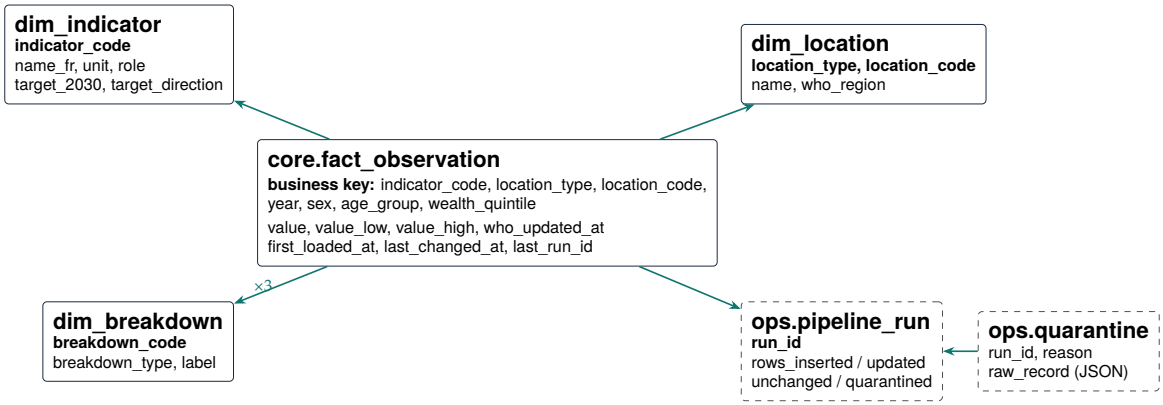


Figure 2: Data model (core and ops schemas; arrows represent foreign keys)

> **Why NULLS NOT DISTINCT?**
>
> By default in SQL, `NULL = NULL` is not true: two "no breakdown" rows are not considered duplicates, so a regular unique constraint would let through endless copies. `UNIQUE NULLS NOT DISTINCT` (PostgreSQL 15+) treats `NULL`s as equal and closes that gap.

Analysis views. `core.v_national` encapsulates the grain rule (both sexes, no wealth breakdown) and `core.v_latest` returns the latest available year. The client's team no longer needs to know these rules to get a correct value.

7 Data quality

7.1 Coverage

All 9 indicators are available for the **47 countries** of the African Region with data (out of 50 country and territory codes). The data is recent: 2023 to 2025 depending on the indicator. The one exception is skilled birth attendance, for which only 32 countries have data after 2020 (median: 2021).

7.2 Threshold calibration: a lesson from real-world data

On the first run, "intuitive" plausibility thresholds sent **25 rows to quarantine**. Reviewing them showed they were not errors, but the statistical traces of major humanitarian crises:

Initial threshold	Rows	What these values were
Under-five mortality ≤ 500 per 1,000	18	Cambodia 1975–1978 (Khmer Rouge regime), South Sudan 1988, Iraq 1942–1944
Maternal mortality ≤ 5,000	7	Rwanda 1994 (genocide), South Sudan 1987–1998 (civil war)

Table 9: Extreme but real values, wrongly quarantined

The thresholds were replaced by **logical limits**: a rate per 1,000 births cannot exceed 1,000, a percentage cannot exceed 100 (ADR 003). Result: **129,368 valid rows, 0 quarantined**, while the checks still block genuine errors, as the tests prove.

> **Lesson**
>
> A quality rule must be calibrated with domain knowledge. The quarantine did its job: isolating suspicious values for human review, instead of silently deleting them or loading them unexamined.

8 Security and governance

Topic	Measure
Secrets	PostgreSQL credentials in `.env` (excluded from Git). A secret-free template, `.env.example`, is versioned. Random password generated locally.
Network exposure	Database reachable from the local machine only (`127.0.0.1:5433`)
Personal data	None: WHO data is aggregated at country level
Data licence	WHO data used under its data policy, for non-commercial purposes, with source attribution
Traceability	SHA-256 manifest, `ops.pipeline_run` log, `last_run_id` on every observation, ADRs, work log

9 Operations

9.1 Commands

```
copy .env.example .env                          # then choose a password
docker compose up -d                            # starts PostgreSQL (port 5433)
pip install -r requirements.txt
python -m src.pipeline                          # extraction + loading + report
python -m src.pipeline --run-date 2026-10-22    # replay a raw batch, no API calls
pytest                                          # 22 tests (5 on a dedicated database)
```

9.2 Production diagnoses

Two real incidents were diagnosed and resolved during the engagement:

Incident	Diagnosis	Resolution
Docker Desktop crashes on start-up	Backend log: two Unix sockets (`sailor-ingest.sock`, `engine.sock`) locked since a hard shutdown a month earlier	Folders renamed (nothing deleted). Docker recreates clean folders.
156 s to load 129,368 rows	Step-by-step timing: `COPY` 0.9 s, upsert 2.8 s, but **connection 130 s**. `localhost` resolves to IPv6 (`::1`) first, where Docker does not listen.	Host `127.0.0.1` and `connect_timeout=10`: pipeline in **10 s** (×15)

Table 10: Incidents resolved

> **Lesson**
>
> Measure instead of guessing: the bottleneck was neither in the Python code nor in PostgreSQL, but in host name resolution.

10 Testing and acceptance

10.1 Automated tests

File	Tests	What is checked
`test_extract.py`	4	OData pagination, failure on unexpected structure, atomic write, retries limited to transient errors
`test_transform.py`	13	Grain (distinct quintiles), breakdowns mapped by type, each quarantine rule, no row lost
`test_load_integration.py`	5	On a real dedicated database: first load, idempotent re-run, revision detected, quarantine stored, unique constraint

Table 11: 22 tests, on a real API extract (Senegal, 2020)

The tests found a **real bug**: in a mixed pandas column, `None` becomes `NaN`, and `NaN` is "truthy" in Python. A single row with an unknown breakdown then sent **every** row to quarantine.

10.2 Fresh-environment acceptance

Copy of the publishable files only, a new Python environment, `.env` recreated from the template, a **blank PostgreSQL database**, and a real extraction from the API.

#	Criterion	Result	
C1	End-to-end run, database included	Install in 57 s, pipeline in 2 min 43 s (real extraction included)	✓
C2	Re-run with no duplicate or change	0 inserted, 0 updated, 129,368 unchanged	✓
C3	Business key covering every dimension	Database constraint, 0 duplicates, integration test	✓
C4	Invalid rows quarantined, never lost	Invariant extracted = loaded + quarantined	✓
C5	No secret in the repository	`.env` excluded, no password in versioned files	✓
C6	Test suite fully green	22 / 22	✓
C7	One query and one answer per question	Section 11	✓

Table 12: Acceptance results (details: `docs/livraison/2026-10-22-pv-recette.md`, in French)

11 Results and recommendations

11.1 Q1 · Where do countries stand against the 2030 targets?

Indicator	Target	Countries on target	Africa median	World
Under-five mortality (per 1,000, 2024)	≤ 25	5 / 47 (11%)	48.7	37.4
Neonatal mortality (per 1,000, 2024)	≤ 12	4 / 47 (9%)	23.0	17.2
Maternal mortality (per 100,000, 2023)	< 70	4 / 47 (9%)	258.2	197.3
DTP3 immunisation coverage (%, 2025)	≥ 90	16 / 47 (34%)	84	85

Table 13: Status against the 2030 targets (latest available year)

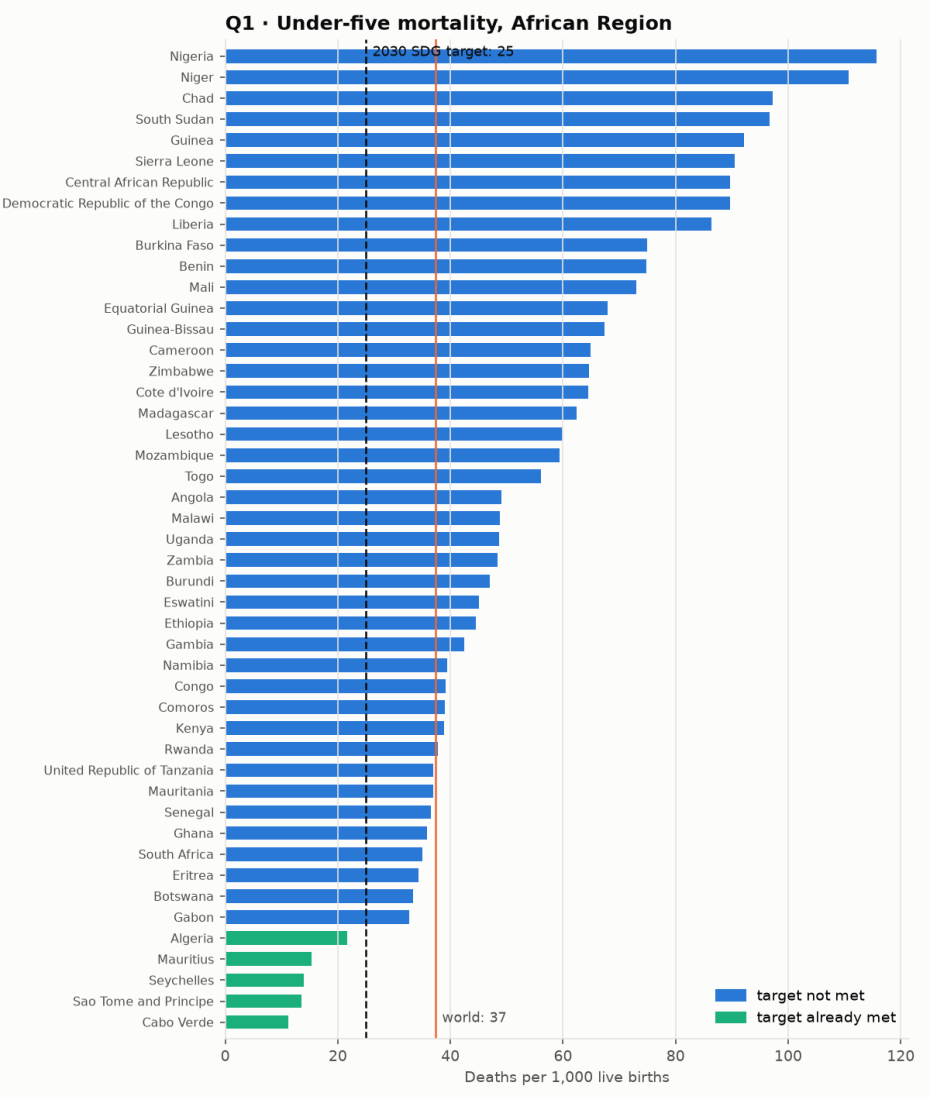


Figure 3: Under-five mortality by African Region country (2024)

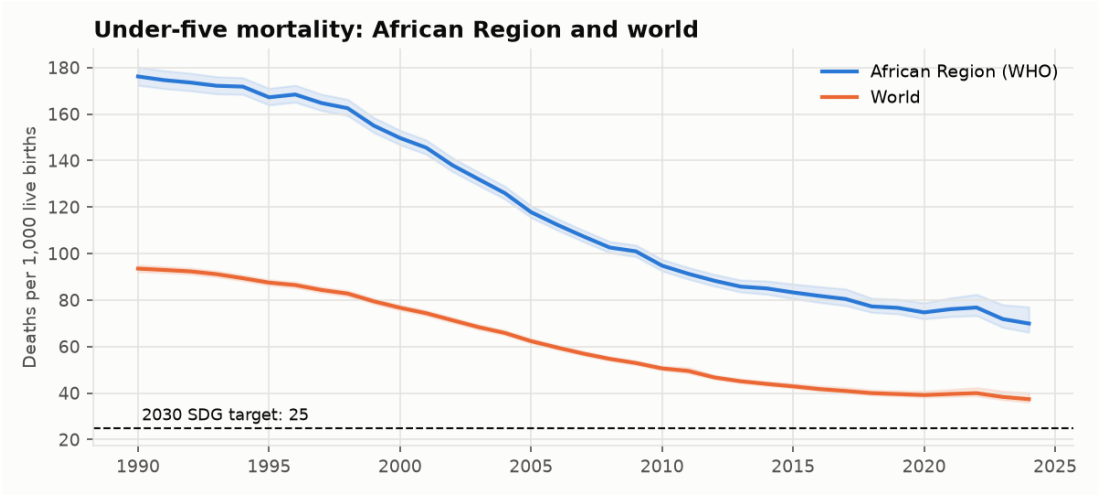


Figure 4: Trend since 1990, with uncertainty intervals

Under-five mortality in the region has fallen 2.5-fold since 1990, from 176 to 70 per 1,000. It remains nearly twice the global average. The regional value (70) is well above the country median (48.7): the most populous countries, such as Nigeria (116), weigh heavily in the regional total.

11.2 Q2 · Which countries will meet the targets by 2030?

Method: the **annual rate of reduction** (ARR) used by the UN Inter-agency Group for Child Mortality Estimation: ARR $= \ln(v_{2015}/v_t)/(t - 2015)$, then a 2030 projection assuming that pace continues.

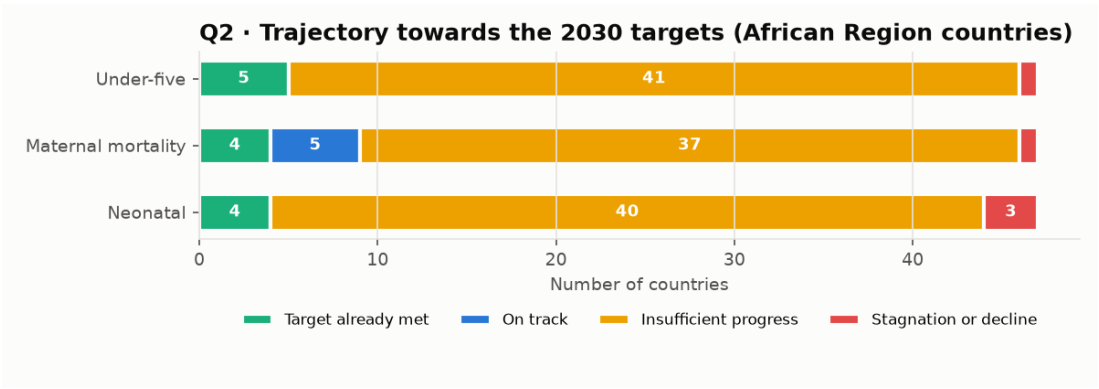


Figure 5: African Region countries by trajectory status

- **Under-five mortality:** no off-target country is on track. The median country reduces mortality by **3.0% a year** whereas **12.4%** would be needed. Zimbabwe is going backwards (59.2 → 64.7 per 1,000 since 2015).
- **Maternal mortality:** the only target with countries on track: Eswatini, Mozambique, Namibia, Sao Tome and Principe and Zambia, with reductions of 7 to 11% a year. Botswana

is going backwards.

- **Neonatal mortality:** the slowest (1.7% a year). It is stagnating or rising in Nigeria, South Africa and Zimbabwe.

11.3 Q3 · How large is the gap between the poorest and richest households?

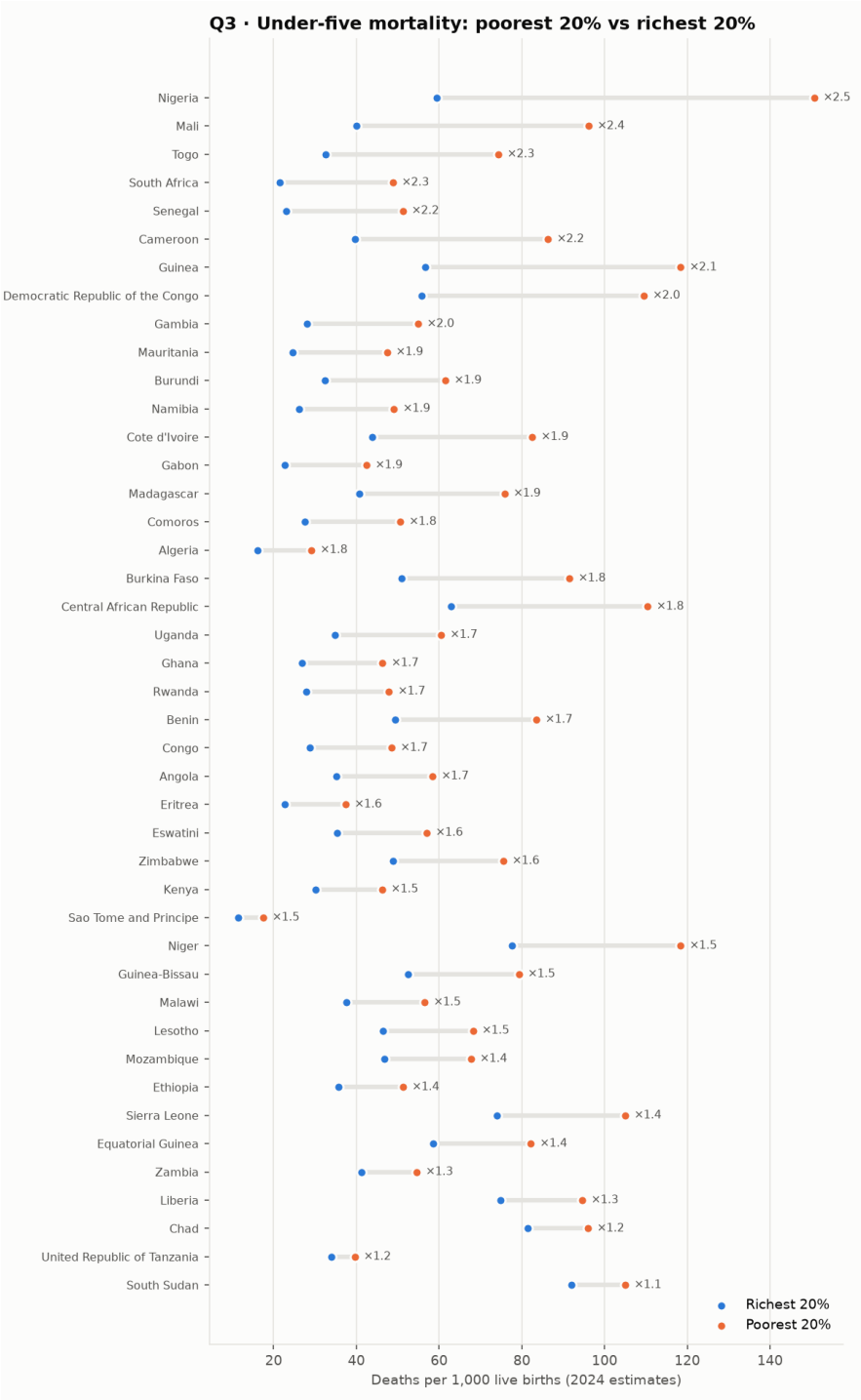


Figure 6: Under-five mortality: poorest 20% vs richest 20% (2024)

Year	Poorest 20%	Richest 20%	Absolute gap	Ratio
1990	186.6	118.8	63.3	1.56
2000	159.7	94.2	50.5	1.49
2015	74.4	46.3	28.2	1.67
2024	61.5	37.6	23.2	1.71

Table 14: Medians across 43 African Region countries (deaths per 1,000 live births)

A child born into the poorest 20% of households is **1.7 times more likely** to die before age 5, and up to 2.5 times in Nigeria and 2.4 times in Mali. **The absolute gap has narrowed** from 63 to 23 per 1,000, but **the relative gap has widened since 2000**: better-off households are improving proportionally faster.

11.4 Q4 · Are inputs associated with outcomes?

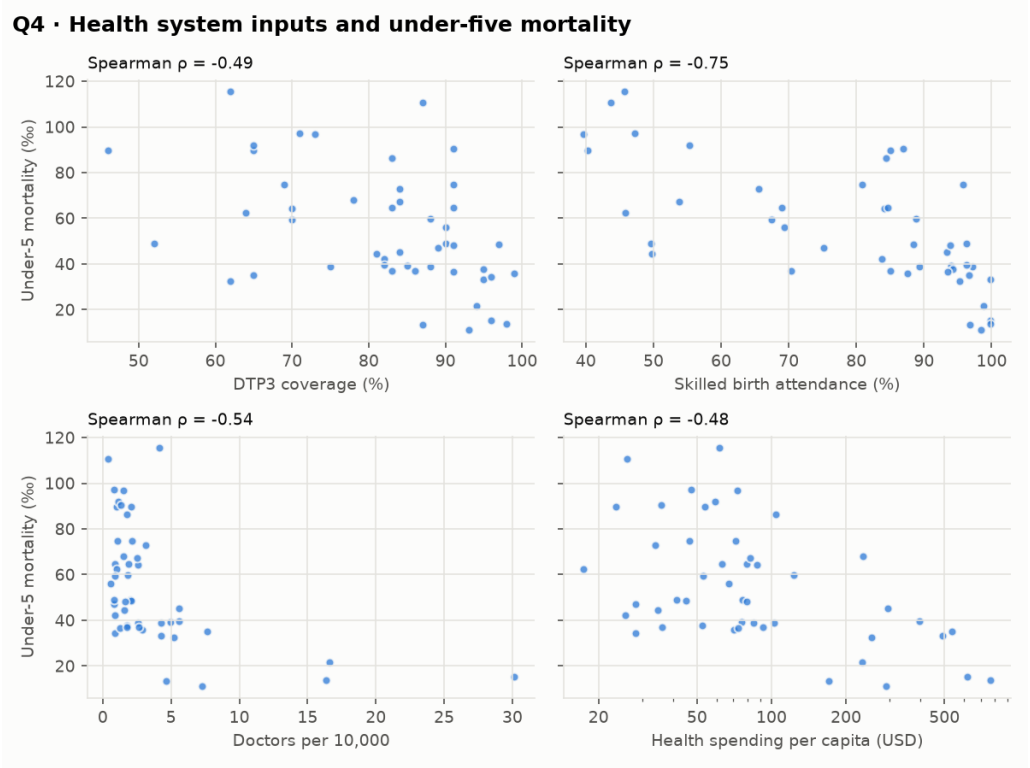


Figure 7: Input indicators and under-five mortality (one dot = one country)

All inputs are associated with lower mortality, but with very different strength: **skilled birth attendance** ($\rho = -0.75$, 45 countries), doctors (-0.54), DTP3 immunisation (-0.49), health spending (-0.48). These are **correlations**, not proof of causation: these inputs also reflect countries' overall level of development.

11.5 Recommendations to the board

For Initiative Baobab Santé

1. **Prioritise neonatal mortality:** it is the slowest-moving target (1.7% a year). Deaths in the first month account for nearly half of under-five mortality (medians: 23.0 out of 48.7 per 1,000).
2. **Explicitly target poor households:** the relative gap is widening. An untargeted programme benefits better-off households first.
3. **Invest in skilled birth attendance:** it is the lever most strongly associated with low mortality in the region.
4. **Study the countries on track** for maternal mortality (Mozambique, Zambia, Eswatini…) to draw lessons.
5. **Re-run the pipeline after every WHO release:** revisions are now applied and tracked automatically.

12 Limitations and technical debt

Limitation	Consequence
WHO data are modelled estimates	Uncertainty intervals are stored but not yet shown for each country
Linear pace projection (ARR)	Assumes the 2015–2024 pace continues until 2030
Correlations (Q4)	They prove no causation
Country names in English	The WHO API does not publish French labels
Technical debt accepted at level 1	
Only the latest value is kept	Full revision history comes in project 2
WHO deletions not handled	An observation withdrawn by WHO would remain in the database
No scheduling	Manual runs (orchestration in project 3)
No continuous integration	Tests are run by hand (CI/CD in project 3)

13 Lessons learned

- **Grain first:** 8 "identical" values hid a dimension. Understanding the grain before modelling avoids the client's mistake.
- **Quality checks must be calibrated:** 25 "outliers" were real historical disasters.
- **Measure, don't guess:** 130 s were lost in DNS resolution, not in the code.

- **Tests find real bugs:** the `None` / `NaN` trap would have emptied the database in production.
- **Check your own assumptions:** wealth data did not come from one-off surveys, and verifying it opened the question of how inequality has evolved.

14 Roadmap

Improvement	Benefit
Revision history	History table (*slowly changing dimension* type 2), planned for project 2
Incremental loading	Download only updated indicators (WHO `Date` field)
Orchestration and CI/CD	Scheduled runs after each release, automatic tests on every change (project 3)
Dashboard	Interactive visualisation for the client's team, connected directly to PostgreSQL

A SQL queries delivered

`00_schema.sql`, `01_views.sql`	Schema (idempotent) and analysis views
`q0_couverture_donnees.sql`	Data coverage in the region
`q1_cibles_2030.sql`	Status against the targets
`q2_rythme_2030.sql`, `q2b_…`	Trajectory by country and summary (ARR method)
`q3_inegalites_richesse.sql`, `q3b_…`	Poor/rich gap, and its evolution since 1990
`q4_moyens_resultats.sql`	Correlations between inputs and mortality
`controle_recette.sql`	Acceptance checks (C2, C3, C4)

B Architecture decision records (ADR)

ADR 001	Containerised PostgreSQL with Docker Compose, secrets in `.env`
ADR 002	Staging and upsert loading, complete `NULLS NOT DISTINCT` business key
ADR 003	Quality checks, quarantine and threshold calibration

C Glossary

SDG 3	UN Sustainable Development Goal 3: “Good health and well-being”
Grain	What one row of a table represents
Staging	Transit area where a batch is loaded before being merged into the reference tables
Upsert	Insert if the row is new, update it if it exists
Quarantine	Table holding rows rejected by the checks, with their reason
ARR	Annual Rate of Reduction of an indicator
Wealth quintile	One fifth of the population ranked by household wealth (Q1: poorest 20%)
Container	Isolated, reproducible environment running software, here PostgreSQL